

# LangChain 架构文档（面向 C++ 开发者）

本文档基于 docs.langchain.com 官方文档编写。读者背景：1 年 C++ 后端开发（多线程、Linux、RPC/Thrift、protobuf、崩溃调试），Python 科研/脚本经验，正在学习 Java/Spring Boot，目标是留日求职。

**更正：** 之前错误地引用了 LangChain4j（社区 Java 移植版）作为参考。正确的参考源是 [docs.langchain.com](https://docs.langchain.com)，LangChain 官方 Python 版。

## 前提：你已经知道的几个概念

如果你跟随之前的 docs/java-learning-notes.md 完成了学习，你已经理解了：

| 概念      | Java 术语                                                  | C++ 类比                                        |
|---------|----------------------------------------------------------|-----------------------------------------------|
| 程序入口    | <code>public static void main(String[] args)</code>      | <code>int main(int argc, char* argv[])</code> |
| 构建工具    | Gradle                                                   | CMake                                         |
| 包管理     | Maven Central                                            | vcpkg                                         |
| 编译产物    | .class 字节码                                               | .o 目标文件                                       |
| 运行时     | JVM                                                      | OS 直接执行                                       |
| 打包产物    | JAR 包                                                    | .exe / .so                                    |
| 框架      | Spring Boot                                              | (C++ 没有直接对应，是一个"管理所有对象的工厂")                   |
| 对象管理    | Spring IoC 容器                                            | 全局对象工厂（自动 new 和管理生命周期）                        |
| 依赖注入    | <code>@Autowired</code>                                  | 连接容器里的实例到这个字段（传引用，不是拷贝）                       |
| 组件标记    | <code>@Component</code> / <code>@Service</code>          | 标记"这个类需要 Spring 自动创建实例"                       |
| HTTP 端点 | <code>@RestController</code> + <code>@PostMapping</code> | 注册 HTTP 路由处理函数                                |
| 日志框架    | SLF4J + Logback                                          | spdlog                                        |

# 一、LangChain4j 是什么

## 一句话解释

LangChain4j 是一个 Java 库，帮你省掉"手动写 Agent 循环"的代码。

## C++ 类比

你在 C++ 后端开发中用过 Thrift RPC——你只需要写 `.thrift` 文件定义接口，Thrift 编译器自动生成 client/server 桩代码，帮你处理序列化、网络传输、请求分发。你只关心业务逻辑。

**LangChain 就是 Java 世界的"Thrift for LLM Agent"**——它帮你处理： - 调用 LLM API（类似 Thrift 处理 RPC 编解码） - 管理对话历史（类似消息队列管理上下文） - 执行工具（类似 protobuf/JSON 反序列化 dispatch 到对应 handler） - 循环调度（类似生产者-消费者模型的调度循环）

## 对比

你的 C++ RPC 服务：

客户端 → Thrift 自动处理 → 你的 handler → Thrift 返回响应

Java LLM Agent（手动写）：

用户消息 → 手动循环（调 API → 解析 → 执行工具 → 循环） → 返回

Java LLM Agent（用 LangChain 思想）：

用户消息 → Agent Executor（自动循环 + 工具 dispatch） → 返回

# 二、当前项目（手动实现）存在的问题

## 当前架构



## 这段代码的问题（从 C++ 角度）

```
// AgentService.java - 手动循环
for (int round = 0; round < 10; round++) {
    // 1. 调用 API（手动构建 JSON 请求体）
    AnthropicClient.AnthropicResponse response = anthropicClient.chat(history);

    // 2. 检查是否有工具调用（手动判断）
    if (response.toolCalls().isEmpty()) {
        return CompletableFuture.completedFuture(response.text());
    }

    // 3. 执行工具（手动 Dispatch）
    for (ToolCall tc : response.toolCalls()) {
        ToolResult result = toolRegistry.execute(tc.name(), tc.input());
        history.add(new Message("user", "工具结果: " + result.content()));
    }
    // 循环回去...
}
```

**类比：**这就像你在 C++ 里不依赖 Thrift 自动生成的桩代码，而是手写 socket 通信 + protobuf 序列化 + 手动 dispatch 每个 RPC 方法。虽然能工作，但多维护一个方法就多一份样板代码。

- 每新增一个工具，要手动注册
- 对话历史的清理/逐出要手动实现
- 流式响应要手动解析 SSE 事件
- 工具调用的参数验证要手动写
- 错误恢复和重试要手动实现

### 三、LangChain4j 的解决方案

#### 核心思想：声明式替代命令式

```
// === 不用 LangChain4j（命令式：你说怎么做） ===
// 你需要手动写：
// 1. 构建 HTTP 请求
// 2. 解析 JSON 响应
// 3. 检查 tool_use
// 4. 执行工具
// 5. 循环
for (int round = 0; round < 10; round++) {
    AnthropicClient.AnthropicResponse response = anthropicClient.chat(history);
    if (response.toolCalls().isEmpty()) return response.text();
    for (ToolCall tc : response.toolCalls()) {
        ToolResult result = toolRegistry.execute(tc.name(), tc.input());
        history.add(new Message("user", "工具结果: " + result.content()));
    }
}

// === 用 LangChain4j（声明式：你说要什么结果） ===
// 只需要定义一个接口，LangChain4j 自动完成上述全部工作
@AiService
public interface CcAgentAiService {
    @SystemMessage("You are an AI coding assistant...")
    String chat(String userMessage); // ← 一行代码，自动处理循环
}
```

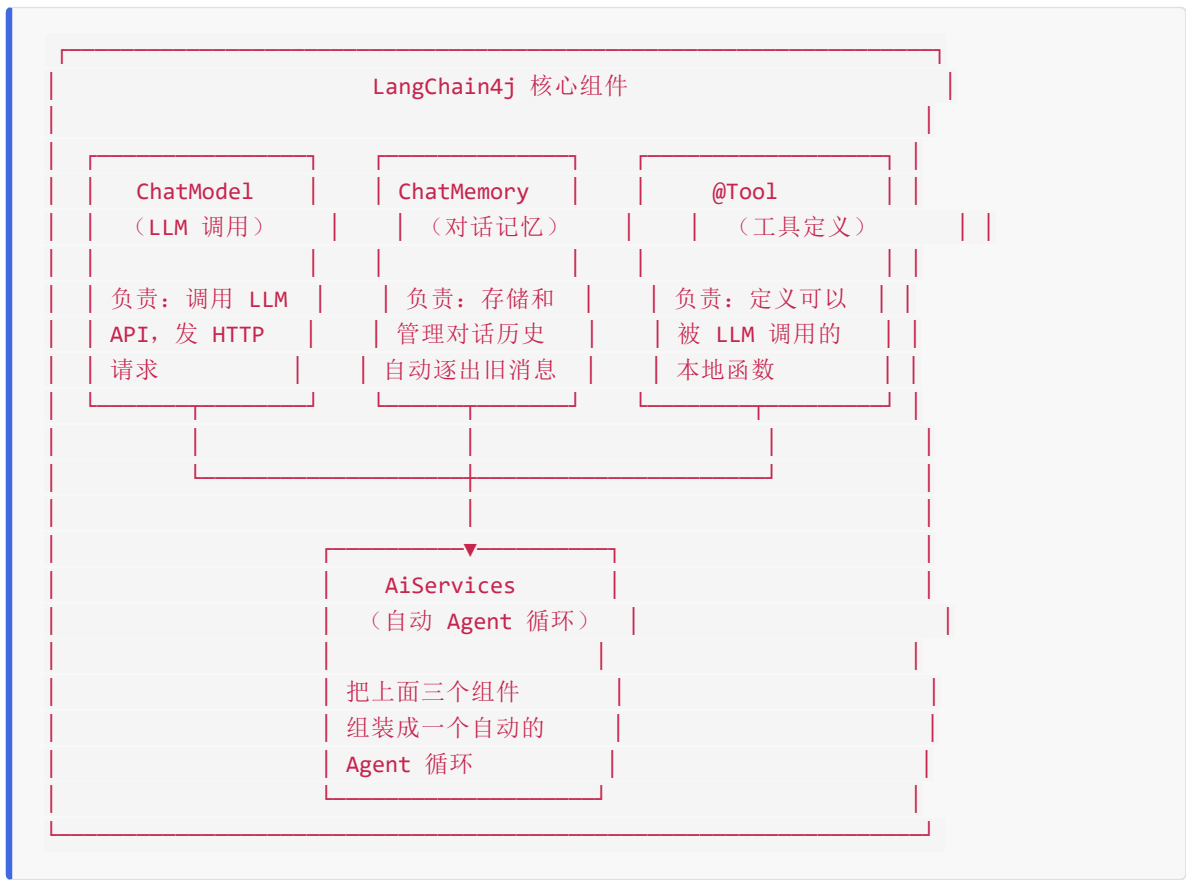
#### 类比 C++

```
// 命令式（手动管理）：手写每个 RPC handler 的 dispatch 逻辑
void process_request(const Request& req) {
    if (req.method == "get_weather") {
        handle_weather(parse_json(req.body));
    } else if (req.method == "search") {
        handle_search(parse_json(req.body));
    }
    // 每加一个方法，要手动加 else if...
}

// 声明式（类比 LangChain 的思路，Thrift 自动生成）：
// .thrift 文件只定义接口，编译器自动生成 dispatch 逻辑
// service WeatherService {
//     string get_weather(1: string city);
//     string search(1: string query);
// }
```

## 四、LangChain4j 的核心组件

组件图



### 4.1 ChatModel — 调用 LLM API

**作用：** 封装和 LLM 的 HTTP 通信，你不需要手动构建 JSON 请求。

```
// 不用 LangChain4j (手动):  
// 1. 构建 Map<String, Object> 请求体  
// 2. 用 HttpClient 发 HTTP POST  
// 3. 解析 JSON 响应  
// 4. 提取 text 和 tool_use  
// → AnthropicClient.java 整个文件  
  
// 用 LangChain4j (自动配置):  
// ChatModel bean 由 Spring Boot 自动创建  
// 只需在 application.yml 里配置 key 和 model  
// chatModel.chat(chatRequest) 一行调用
```

**类比：** ChatModel 就像 Thrift 生成的 Client 桩——你只需要调用 `client.GetWeather(city)`，它自动处理序列化、网络传输，你不需要关心底层的 socket 通信和 protobuf 编码。

## 4.2 ChatMemory — 管理对话历史

**作用：** 自动管理对话历史，支持逐出旧消息（超过 token 限制时自动删除最早的）。

```
// 不用 LangChain4j（手动）：  
// List<Message> history = new ArrayList<>(); // 手动创建列表  
// history.add(new Message("user", message)); // 手动添加  
// // 没有自动逐出机制，历史越长 token 越多  
  
// 用 LangChain4j（自动）：  
// ChatMemoryProvider 自动创建和管理  
// TokenWindowChatMemory 自动限制总 token 数  
// 每次对话自动维护历史
```

**类比：** ChatMemory 就像你在 C++ 后端里用线程安全的生产者-消费者队列（有容量限制，满了自动丢弃旧数据），你只管往队列里 push，不需要关心内存管理。

## 4.3 @Tool 注解 — 定义可被 LLM 调用的函数

**作用：** 用注解标记 Java 方法，让 LLM 可以调用。注解里的描述信息会被自动转换成 API 需要的 JSON Schema。

```
// 不用 LangChain4j（手动定义工具）：
@Component
public class BashTool implements AgentTool { // 实现接口
    @Override public String getName() { return "bash"; }
    @Override public String getDescription() { return "执行 shell 命令"; }
    @Override public String execute(Map<String, Object> input) throws Exception {
        String command = (String) input.get("command"); // 手动取参数
        // ...
    }
}

// + 需要 AgentTool 接口
// + 需要 ToolRegistry 类来收集和 Dispatch
// + 需要手动从 Map 中取参数并类型转换
// + 需要手动编写 JSON Schema 给 LLM

// 用 LangChain4j（注解驱动）：
@Component
public class BashTool {
    @Tool("执行 shell 命令")
    public String runCommand(@P("要执行的命令") String command) throws Exception {
        // command 自动从 LLM 的参数中解析，类型安全
        // ...
    }
}

// + 不需要接口
// + 不需要 ToolRegistry (@Tool 注解自动被发现)
// + 参数自动类型转换
// + JSON Schema 自动从方法签名生成
```

**类比 C++：** @Tool 就像 RPC 框架里的接口定义（IDL）——你定义函数签名和注释，框架自动生成序列化/反序列化代码。

| C++ 概念           | Java @Tool        |
|------------------|-------------------|
| gRPC 的 .proto 文件 | @Tool 注解          |
| protobuf 序列化     | 自动 JSON Schema 生成 |
| gRPC stub        | AiServices 代理     |

4.4 AiServices — 自动 Agent 循环

**作用：** 把 ChatModel + ChatMemory + @Tool 组装成一个自动的 Agent 循环。

**AiServices** 内部做的事情（自动的）：

1. 从 **ChatMemory** 加载历史
2. 构建 **ChatRequest**（系统提示词 + 历史 + 工具列表）
3. 调用 **ChatModel**
4. 如果响应包含 **toolExecutionRequests**：
  - 执行对应的 **@Tool** 方法
  - 添加 **ToolExecutionResultMessage** 到历史
  - 回到步骤 2
5. 如果响应是纯文本：
  - 保存到 **ChatMemory**
  - 返回最终结果

**类比：** **AiServices** 就像 Thrift 自动生成的 RPC Server 处理器——你只注册 handler 函数，它内部自动处理请求接收、反序列化、dispatch、错误恢复。你只需要关心"这个 RPC 收到什么参数、返回什么结果"。

## 五、新架构 vs 旧架构

### 旧架构（手动）

```
ChatController
├── AgentService (350 行手动循环代码)
│   ├── List<Message> history (手动管理)
│   ├── AnthropicClient (手动 HTTP 调用 + JSON 解析)
│   ├── ToolRegistry (手动工具注册 + Dispatch)
│   └── for (int round = 0; round < 10; round++) { ... }
```

**需要手动维护的代码：** - **AgentService.java** — 循环逻辑 - **AnthropicClient.java** — HTTP 客户端 - **ToolRegistry.java** — 工具注册表 - **AgentTool.java** — 工具接口 - **Message.java** — 消息模型 - **ToolCall.java** — 工具调用模型 - **ToolResult.java** — 工具结果模型

### 新架构（LangChain4j）

```
ChatController
├── CcAgentAiService (@AiService 接口, 10 行)
│   ├── ChatModel (自动配置, 0 行代码)
│   ├── ChatMemory (自动管理, 0 行代码)
│   └── @Tool 方法 (修饰现有工具类, +3 行/工具)
```

**可以删除的代码：** - **AgentService.java** — 删除（循环由 **AiServices** 自动完成） - **AnthropicClient.java** — 删除（**ChatModel** 自动配置） - **ToolRegistry.java** — 删除（**@Tool** 自动发现） - **AgentTool.java** — 删除（注解取代接口） - **Message.java** — 删除（**LangChain4j**



内置类型) - `ToolCall.java` — 删除 (LangChain4j 内置类型) - `ToolResult.java` — 删除 (LangChain4j 内置类型)

## 六、数据流详解

### 6.1 完整数据流

用户在浏览器/curl 输入: "帮我读 main.java"

↓ HTTP GET /api/chat/v2?message=帮我读main.java

`ChatController.chatV2()`

↓ 调用

`@AiService` 代理 (Spring Boot 自动创建的)

↓ 代理内部自动执行:

1. `ChatMemoryProvider.getMemory(sessionId)`

↳ 返回 `TokenWindowChatMemory` (最多 8K tokens)

2. 构建 `ChatRequest`:

└─ `SystemMessage`: "You are an AI coding assistant..."

└─ `messages`: ...历史消息...

└─ `toolSpecifications`: [bash, file\_read, file\_write]

3. `ChatModel.chat(request)`

↓ HTTP POST `https://api.anthropic.com/v1/messages`

↳ Claude 返回: `tool_use(name="file_read", input={path: "main.java"})`

4. AI Service 检测到 `toolExecutionRequests`

↳ 找到 `@Tool` 方法: `FileReadTool.readFile(path)`

↳ 执行: `readFile("main.java")`

↳ 返回: 文件内容字符串

5. 添加 `ToolExecutionResultMessage` 到 `ChatMemory`

6. 再次调用 `ChatModel.chat(request)` (带工具结果)

↳ Claude 返回: 纯文本 "文件内容是..."

7. 添加到 `ChatMemory`, 返回结果给用户

## 6.2 C++ 类比

```
// C++ 类比（不是真实代码，帮助理解数据流）：
struct Pipeline {
    Memory* memory;      // 对话历史
    ModelClient* model;  // LLM 调用
    ToolRegistry* tools; // 工具函数表
};

string run(string input) {
    memory->add(UserMessage(input));

    while (true) {
        auto request = build_request(memory->get(), tools->schemas());
        auto response = model->chat(request);

        if (response.tool_calls.empty()) {
            memory->add(AiMessage(response.text));
            return response.text;
        }

        for (auto& tc : response.tool_calls) {
            auto result = tools->execute(tc.name, tc.input);
            memory->add(ToolResultMessage(result));
        }
    }
}
```

Pipeline 就是 LangChain4j 的 `AiServices` 内部实现。你不需要自己写这个循环。

---

## 七、新增的组件详解

### 7.1 @AiService — 声明式接口

```
// 这个接口不需要实现类！
// Spring Boot 启动时自动创建代理对象
@AiService
public interface CcAgentAiService {

    // @SystemMessage = 系统提示词（类似 C++ 的 const string 配置）
    // 每次调用 chat() 时自动注入到请求里
    @SystemMessage("""
        You are an AI coding assistant running on the user's local machine.
        You have access to tools for reading files, writing files,
        and running shell commands.
        """)
    String chat(String userMessage);

    // chatStream = 流式版本，返回 TokenStream
    // TokenStream 类似 protobuf 反序列化流，逐个字段解析，不等整条消息到齐
    @SystemMessage("""
        You are an AI coding assistant running on the user's local machine.
        """)
    TokenStream chatStream(String userMessage);
}
```

**关键点：** - 你不需要写实现类（不像 C++ 需要写 .cpp 实现 .h 声明的函数） - Spring 自动创建代理对象（类似 C++ 的动态代理/虚函数表，但是自动的） - 方法名和参数由你定义，返回值由 LangChain4j 决定（String 或 TokenStream）

## 7.2 @Tool — 可被 LLM 调用的函数

```
@Component // 标记为 Spring 管理的组件
public class BashTool {

    // @Tool("描述") = 告诉 LLM 这个工具能做什么
    // LLM 会看到: "执行 shell 命令。例如: git status、ls、npm install 等。"
    @Tool("执行 shell 命令。例如: git status、ls、npm install 等。")
    // @P("描述") = 参数的说明 (帮助 LLM 理解参数含义)
    // LLM 会看到参数描述: "要执行的 shell 命令"
    public String runCommand(@P("要执行的 shell 命令") String command) throws Exception {
        // command 的值由 LangChain4j 自动从 LLM 的 tool_use 参数中提取并类型转换
        Process process = Runtime.getRuntime().exec(
            new String[]{"bin/sh", "-c", command});
        String output = new String(process.getInputStream().readAllBytes());
        int exitCode = process.waitFor();
        if (exitCode != 0) {
            String error = new String(process.getErrorStream().readAllBytes());
            throw new RuntimeException("命令失败: " + error);
        }
        return output;
    }
}
```

**LangChain4j 自动做的事情：**

1. 从方法签名生成 JSON Schema :  
{"name": "runCommand", "parameters": {"command": {"type": "string"}}}
2. 从 LLM 的 tool\_use 参数中提取值: {"command": "ls -la"} → command = "ls -la"
3. 自动调用 runCommand("ls -la")
4. 把返回值包装成 ToolExecutionResultMessage

## 7.3 TokenStream — 流式响应

```
// TokenStream 表示"还在一直接受数据的流"
// 类比 C++: 类似 std::future<std::string>, 但是可以逐个收到结果
TokenStream stream = service.chatStream("帮我读文件");

// 注册回调: 每收到一个 token 就调用
stream.onPartialResponse(token -> {
    System.out.print(token); // 实时输出到屏幕
});

// 注册完成回调
stream.onCompleteResponse(fullText -> {
    System.out.println("\n完成!");
});

// 启动流式处理 (非阻塞, 后台线程处理)
stream.start();
```

### 类比 C++ RPC 异步处理:

```
// C++ RPC 异步回调模式
rpc_client->call_async("get_weather", city, [](const Response& resp) {
    handle(resp); // 每收到一个响应就处理
});
```

## 八、迁移路线图

### Phase 1: 共存 (新旧代码并行)

|                       |                              |
|-----------------------|------------------------------|
| build.gradle          | ← 加两个 LangChain4j 依赖         |
| application.yml       | ← 加 langchain4j 配置段          |
| BashTool.java         | ← 加 @Tool 方法 (保留原有代码)        |
| FileReadTool.java     | ← 加 @Tool 方法                 |
| FileWriteTool.java    | ← 加 @Tool 方法                 |
| CcAgentAiService.java | ← 新建 @AiService 接口           |
| ChatController.java   | ← 新增 /api/chat/v2 端点 (旧端点保留) |

### Phase 2: 验证

|                 |           |
|-----------------|-----------|
| 测试 /api/chat    | ← 旧端点正常工作 |
| 测试 /api/chat/v2 | ← 新端点正常工作 |

### Phase 3: 清理

删除: AgentService.java、AnthropicClient.java、ToolRegistry.java  
删除: AgentTool.java、Message.java、ToolCall.java、ToolResult.java  
简化: AgentProperties.java (删除已迁移到 langchain4j 配置的字段)

### Phase 4: 默认

/api/chat 指向新实现, 去掉 v2 后缀

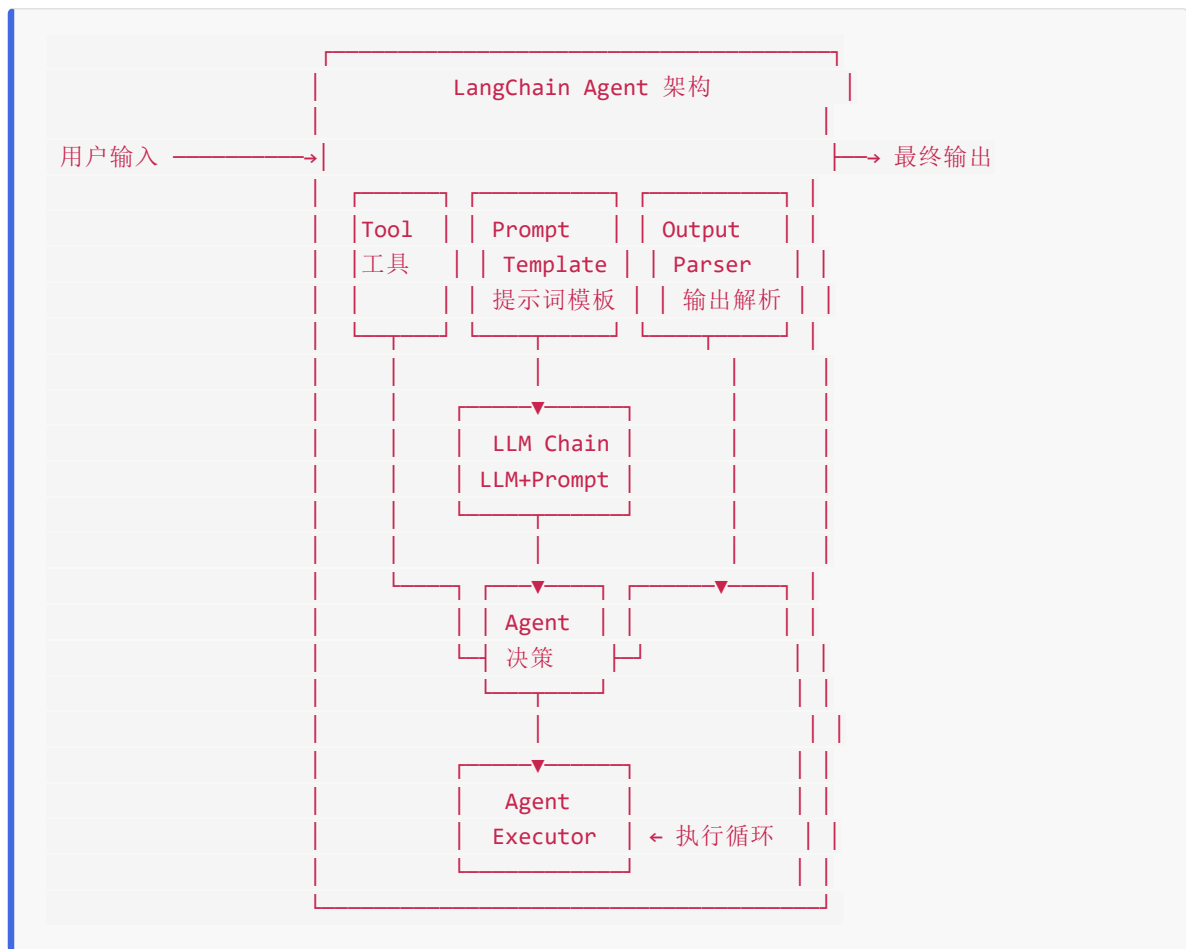
九、Java 概念速查表

| Java 概念           | 代码                                                  | C++ 类比                                                                 |
|-------------------|-----------------------------------------------------|------------------------------------------------------------------------|
| 接口                | <code>interface Foo { void bar(); }</code>          | 纯虚类 <code>class Foo { virtual void bar() = 0; }</code>                 |
| 注解                | <code>@GetMapping("/api")</code>                    | 编译期属性 <code>[[nodiscard]]</code>                                       |
| 方法签名              | <code>String foo(int x)</code>                      | <code>std::string foo(int x)</code>                                    |
| 泛型                | <code>List&lt;String&gt;</code>                     | <code>std::vector&lt;std::string&gt;</code>                            |
| Record            | <code>record Point(int x, int y) {}</code>          | <code>struct Point { int x; int y; };</code> (自动生成 getter/hash/equals) |
| Lambda            | <code>x -&gt; x + 1</code>                          | <code>[](int x) { return x + 1; }</code>                               |
| 方法引用              | <code>BashTool::runCommand</code>                   | <code>&amp;BashTool::runCommand</code>                                 |
| Stream API        | <code>list.stream().filter(x -&gt; x &gt; 0)</code> | <code>std::ranges::filter(list, [](int x) { return x &gt; 0; })</code> |
| CompletableFuture | <code>future.thenApply(x -&gt; x + 1)</code>        | <code>std::future</code> + callback (但 Java 的更好用)                      |
| SseEmitter        | <code>emitter.send("data")</code>                   | 服务器端推送事件的句柄                                                            |
| @Component        | 标记类                                                 | 告诉 Spring: "自动 new 这个类的实例"                                             |
| @Autowired        | 标记字段                                                | 告诉 Spring: "把容器里的实例连接到这个字段"                                            |

十、LangChain v1 核心架构深度解析

基于 docs.langchain.com + OpenAI Cookbook 的真实实现。

## 10.1 架构总览：六个组件，一个循环



10.2 六个组件的本质

| # | 组件              | 本质                                         | C++ 类比            | 在 LangChain v1 中的 API                                              |
|---|-----------------|--------------------------------------------|-------------------|--------------------------------------------------------------------|
| 1 | Tool            | 被 LLM 调用的函数，带名称、描述、参数 schema               | 函数指针 + 元数据 struct | <code>@tool</code> 装饰器或 <code>Tool(name, func, description)</code> |
| 2 | Prompt Template | 控制 LLM 行为的提示词，定义"思考→行动→观察"格式               | 格式化字符串 sprintf    | <code>ChatPromptTemplate</code> 或字符串                               |
| 3 | Output Parser   | 把 LLM 的文本输出解析成结构化数据 (Action/Final Answer)  | 正则解析器 + 状态机       | 正则提取 <code>Action: xxx / Final Answer: xxx</code>                  |
| 4 | LLM Chain       | LLM + PromptTemplate 的组合，输入→LLM推理→输出       | 带配置的推理引擎          | <code>LLMChain(llm, prompt)</code>                                 |
| 5 | Agent           | 组装 LLM Chain + Output Parser + Tools 的决策引擎 | 决策状态机             | <code>LLMSingleActionAgent(llm_chain, output_parser, tools)</code> |
| 6 | Agent Executor  | 运行"思考→行动→观察"循环的调度器                         | 事件循环 + 回调         | <code>AgentExecutor.from_agent_and_tools()</code>                  |

10.3 从输入到输出的完整数据流

下面跟踪一条完整请求：用户问"加拿大有多少人口？"

**进程/线程说明：** LangChain 在 Python 中是单线程同步执行的。所有组件都在**同一个进程、同一个线程**中运行。但在 Java/Spring Boot 版本中，`@Async` 会让 AgentService 在单独的线程池线程中执行。





Observation: "39,566,248"

AgentExecutor 把 (AgentAction, Observation) 记入 intermediate\_steps

### 第 3 步: AgentExecutor 调用 Agent (第 2 次, 带 Observation)

AgentExecutor 把原问题 + intermediate\_steps 传给 Agent

```
Agent (第 2 轮)
输入: Question + 之前的 Thought/Action/Observation

LLM 输出:
"""
Thought: I now know the answer
Final Answer: The population of Canada as of
2023 is 38,664,637.
"""

OutputParser.parse(llm_output):
→ 检测到 "Final Answer:"
→ 返回 AgentFinish(output="38,664,637")
```

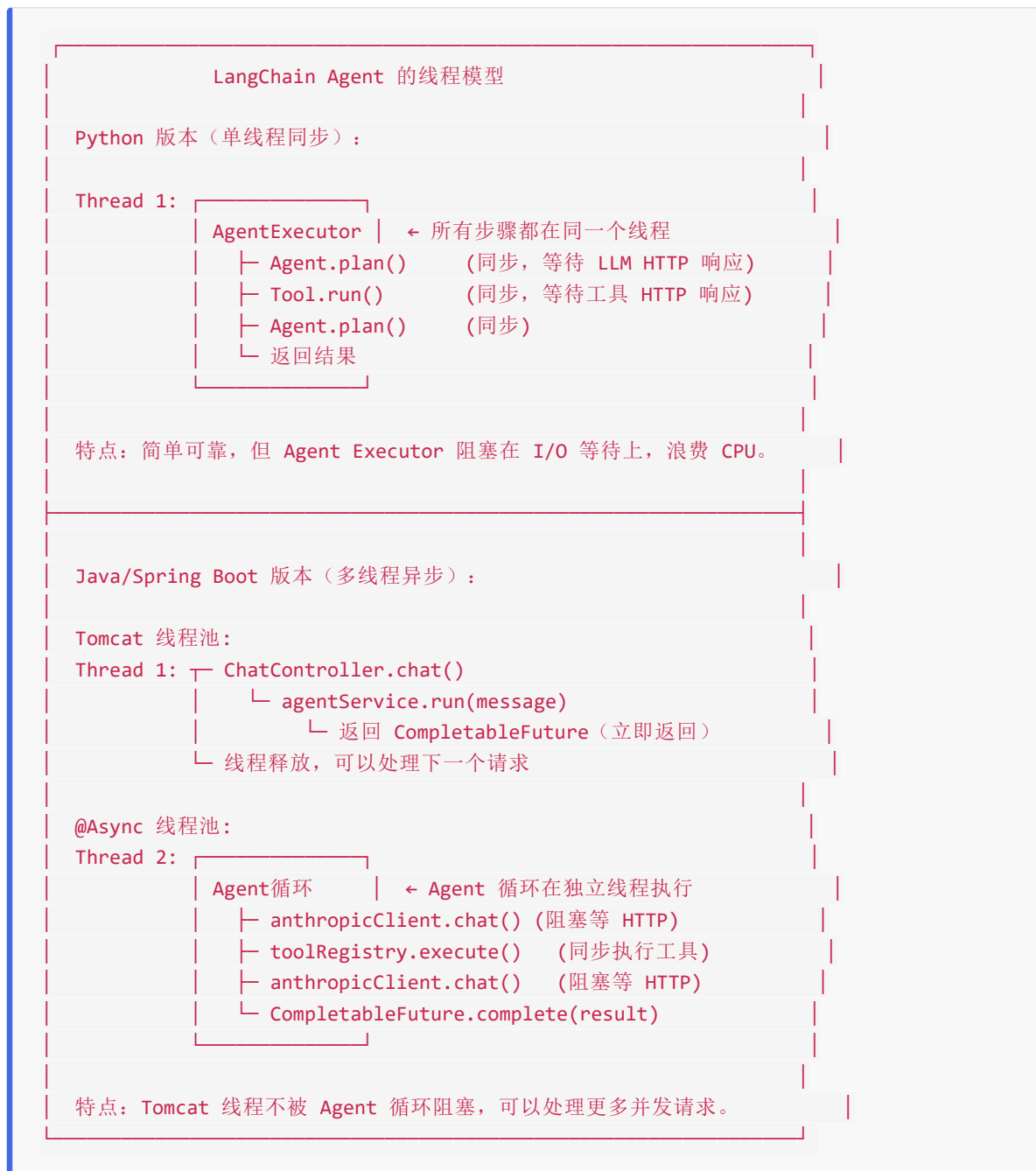
Agent 返回: AgentFinish

### 第 4 步: AgentExecutor 返回最终结果

AgentExecutor 看到 AgentFinish, 退出循环

返回: "The population of Canada as of 2023 is 38,664,637."

## 10.4 线程/进程模型



## 10.5 LangChain v1 ( create\_agent ) 的简化

LangChain 官方在 v1 版本中将上述六个组件简化为一个函数：

```
# LangChain v1 (docs.langchain.com 最新版)
from langchain.agents import create_agent

agent = create_agent(
    model="claude-sonnet-4-6",      # LLM Chain 被内化为 model 参数
    tools=[get_weather, calculator], # Tools 直接传入
    system_prompt="You are...",     # Prompt Template 被内化为 system_prompt
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "What's the weather?"}]}
)
```

### 简化了什么：

|                                   |                                                        |
|-----------------------------------|--------------------------------------------------------|
| 旧版（显式，学习用）：                       | 新版（隐式，生产用）：                                            |
| PromptTemplate + LLMChain + Agent | <code>create_agent(model, tools, system_prompt)</code> |
| OutputParser                      | → model 参数（内部自动组装 LLM Chain）                           |
| AgentExecutor（手动循环）               | → 内建，开发者看不到                                            |
|                                   | → <code>agent.invoke()</code> 内部自动循环                   |

但内部做的事情完全一样 —— `create_agent` 内部仍然有 Output Parser 解析 LLM 输出、Agent Executor 循环调用工具。

10.6 对比表：LangChain vs 我们的 Java Agent

| 步骤     | LangChain (Python)                                                                                | 我们的 Java Agent                                                       |
|--------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| 接收输入   | <code>agent_executor.run(input)</code>                                                            | <code>ChatController.chat(request)</code>                            |
| 组装提示词  | <code>PromptTemplate.format()</code>                                                              | 在 <code>AnthropicClient</code> 里手动构建 Map                             |
| 调用 LLM | <code>LLMChain.run(prompt)</code> → <code>ChatOpenAI</code>                                       | <code>AnthropicClient.chat(history)</code> → <code>HttpClient</code> |
| 解析输出   | <code>OutputParser.parse(llm_output)</code> → <code>AgentAction</code> / <code>AgentFinish</code> | 手动解析 JSON <code>response.toolCalls()</code>                          |
| 执行工具   | <code>tool.run(action_input)</code>                                                               | <code>ToolRegistry.execute(name, input)</code>                       |
| 循环控制   | <code>AgentExecutor._take_next_step()</code> 自动循环                                                 | <code>for (int round = 0; round &lt; 10; round++)</code>             |
| 返回结果   | <code>AgentFinish.return_values["output"]</code>                                                  | <code>CompletableFuture.completedFuture(text)</code>                 |
| 记忆管理   | <code>ConversationBufferWindowMemory(k=2)</code>                                                  | <code>List&lt;Message&gt; history</code>                             |
| 流式输出   | <code>agent.stream()</code>                                                                       | <code>SseEmitter</code>                                              |

10.7 C++ 开发者的理解要点

1. LangChain 不是一个"服务"或"进程"，它是一个库 (library) 。

- 你安装 `pip install langchain`，然后在你的代码里 `import` 它
- 它在你的进程里运行，和你的代码在同一线程
- **不是** 一个单独的服务器或微服务

2. Agent 循环本质上是一个 while 循环。

```

// LangChain 的 Agent Executor 本质上就是这个 C++ 伪代码
struct AgentResult {
    bool is_final;
    string output;
    string action;
    string action_input;
};

string run_agent(string input, vector<Tool>& tools, LLM& llm) {
    vector<pair<AgentAction, string>> steps;

    while (true) {
        // 1. 组装提示词 (PromptTemplate)
        string prompt = format_prompt(input, tools, steps);

        // 2. 调 LLM (LLMChain)
        string llm_output = llm.chat(prompt);

        // 3. 解析输出 (OutputParser)
        auto result = parse_output(llm_output);

        if (result.is_final) {
            return result.output; // AgentFinish
        }

        // 4. 执行工具
        string observation = tools[result.action].run(result.action_input);

        // 5. 记录步骤
        steps.push_back({result.action, observation});

        // 6. 循环回去 (带 Observation 的上下文)
    }
}

```

### 3. `create_agent v1` 就是这个 `while` 循环的包装。

所有复杂的东西 (Prompt Template、Output Parser、LLM Chain、Tool 执行) 都被封装在 `create_agent()` 内部。你只需要传三个参数: 模型、工具列表、系统提示词。